Guía de integración del dataset BES-jun con el futuro sistema Asesor ESP

Esta guía es para quien vaya a conectar una aplicación con este expediente y no participó en el análisis. El cálculo ya está hecho en Python. La aplicación no debe reimplementar las ecuaciones.

Qué contiene el dataset

BES-jun es un libro de cinco hojas. Cada hoja es un punto de operación de una bomba electrosumergible: presiones, caudal, frecuencia, profundidades, una tabla de bomba de seis puntos a 60 Hz y un survey de trayectoria. No es una serie temporal. No es un catálogo de fabricante.

El archivo analizado es Copia de FINAL BES-jun.xlsx. La copia de trabajo, de solo cálculo, es `entrada/BES_jun.xlsx`. Su huella está en `manifiesto.json`.

Qué pozos identifica

La identificación es provisional. En cada hoja el rótulo Cluster coincide con el nombre de la hoja y el rótulo Pozo es otro código. El libro no dice cuál es la llave. Dentro de este archivo cada código aparece una sola vez, así que el motor puede resolver la hoja. No debe fusionarse con un pozo de otro dataset solo porque el texto coincida, hasta que alguien confirme la llave.

Pozo (rótulo)	Hoja	Cluster	Bomba escrita	Identidad
QF-084	QF-554H	QF-554H	S 8000N	provisional
QF-301	QF-545H	QF-545H	S 8000N	provisional
QF-147	QF-500H	QF-500H	S 8000N	provisional
QF-005	QF-378HST	QF-378HST	S6000N	provisional
QF-306	QF-549HST	QF-549HST	S6000N	provisional

Qué modelos hay

Tres, versión `transcripcion-1`, en `modelos.json`:

- `M-BES-FLUIDO`: gravedad, densidad y gradiente, como los escribe la hoja.
- `M-BES-BALANCE`: PIP, THP, fricción y THD.
- `M-BES-INTERSECCION`: caudal en el que la altura de la bomba iguala la THD.

Reproducen la caché de la hoja. Eso no aprueba la física. El signo del PIP, el exponente de la fricción y la unidad de BS&W tienen consultas abiertas.

Qué reglas hay y cuáles están pendientes

`reglas.json` trae RC-BES-01, RC-BES-02, RC-BES-03 y RC-BES-04.

- RC-BES-01: caudal por encima del máximo escrito, ambos en la misma frecuencia.
- RC-BES-02: caudal por debajo del mínimo escrito, en la misma frecuencia.
- RC-BES-03: caudal entre esos dos límites, en la misma frecuencia.
- RC-BES-04: cercanía a la máxima eficiencia. **No evaluable.** No hay curva de fabricante.

Las tres primeras están en `pendiente_validacion`. Ninguna está aprobada. En las cinco hojas, con los datos registrados, la regla que se activa es RC-BES-03. En QF-084 el caudal es 10301 BFPD a 63 Hz y el máximo a 60 Hz es 10000 BFPD; escalado, el máximo es 10500 BFPD. Comparar el caudal de operación con el máximo de 60 Hz es un artefacto de la primera versión y no debe volver a mostrarse como la clasificación vigente. El escalamiento no valida el límite ni la afinidad.

Qué variables hace falta

`variables.json` clasifica cada una. Para evaluar el punto ya registrado no hay que volver a teclear los parámetros de la hoja: están en `pozos.json`.

La aplicación pide una variable solo cuando el operador está corrigiendo una medición o cuando falta en la configuración. Si la envía, declara nombre, valor, unidad y procedencia (`configuracion_del_pozo`, `operador`, `sensor` o `no_declarada`).

En modo operacional el operador puede enviar caudal, frecuencia, presiones, temperatura, °API y BS&W, cada uno una sola vez y con su unidad canónica: caudal en BFPD, frecuencia en Hz, presiones en psi, temperatura en °C, BS&W como fracción, °API en °API. Límites, etapas, diámetro, profundidades, frecuencia de la curva, referencia de bomba y la tabla de seis puntos salen de la ficha del pozo. Enviarlos como medición produce `PARAMETRO_PROTEGIDO` y no se calcula. Una variable repetida, aunque los dos valores coincidan, produce `VARIABLE_DUPLICADA`: no se usa el primero ni el último. Un JSON con la misma clave dos veces se rechaza antes de construir el diccionario (`CLAVE_JSON_DUPLICADA`).

Un escenario hipotético pide `modo = simulacion`, un `escenario_id` y los cambios en `parametros_escenario`. El resultado lleva `hipotetico: true`, no actualiza la ficha y no autoriza una recomendación. La curva del escenario es una lista de al menos dos puntos, cada uno solo con `caudal_bfpd` y `head_ft` numéricos y finitos, en BFPD y ft, sin campos extra. En el orden enviado el caudal tiene que ser estrictamente creciente y el head estrictamente descendente. Es el contrato de esta representación, no una ley de todas las bombas. No se reordena, no se redondea para declarar un duplicado y no se elimina ningún punto. Un número suelto, un punto incompleto o un campo inesperado se rechazan como `entrada_invalida` y no llegan al cálculo. Un parámetro numérico del escenario solo admite `valor` y `unidad` canónica; no hay conversión. `fecha_observacion` es `AAAA-MM-DD` o esa fecha con hora ISO 8601. Si hay hora, puede llevar `Z` o `±HH:MM`. No se cambia de zona y no se usa la fecha `=TODAY()` del Excel. Los identificadores van sin espacios exteriores.

El contrato 1.3.0 rechaza la solicitud —estado `entrada_invalida`, `dataset_evaluado` nulo, cero reglas— si falta la versión, falta la unidad, el campo no existe, donde va un número llega un texto, el dataset no es BES-jun o se intenta usar la ficha de otro pozo. La salida distingue `dataset_solicitado` de `dataset_evaluado`. Un dataset ajeno no queda registrado como si se hubiera procesado.

`frecuencia_hz` es la frecuencia registrada. El rango observado de las cinco hojas no es el rango autorizado. El límite autorizado de frecuencia no está en el libro y el catálogo lo deja en null.

Cómo se invoca el motor

Desde la raíz del expediente, con el entorno del proyecto:

```
PYTHONPATH=src /home/antonio/projects/HUC/.venv/bin/python -m evaluacion.motor --dataset BES-jun --pozo QF-084
```

En código:

```
from evaluacion.motor import evaluar, evaluar_solicitud
evaluar("BES-jun", "QF-084")
evaluar_solicitud(solicitud)
```

No hace falta Jupyter ni una pantalla. No llama a la red. No llama a un modelo de lenguaje.

Cómo se lee el resultado

`estado_evaluacion` vale:

- `completada_preliminar`: se calculó y se aplicó la regla candidata. El resultado es preliminar.
- `no_evaluable`: falta un dato de la ficha, el pozo no está, la identidad es ambigua, el modelo no calculó o la regla no puede ejecutarse. No hay un sí ni un no forzados.
- `entrada_invalida`: la solicitud no pasó el esquema, la unidad, el dominio, la unicidad o la protección de la ficha. `errores` lista campo, código y detalle. `dataset_evaluado` es nulo y no se llama a los modelos.
- `error_interno`: la solicitud era válida y el modelo falló después. No es una entrada inválida y no es un resultado operacional.

Cada elemento de `reglas` trae `id`, `estado_regla`, `resultado`, `evaluable`, `aprobada_para_uso_operacional`, `evidencia` y `limitaciones`. La evidencia numérica está en `evidencia`, no en el texto del mensaje.

`recomendacion_operacional_autorizada` es falso mientras no exista una regla aprobada para ese uso. La aplicación no debe convertir una advertencia preliminar en una orden de bajar o subir la frecuencia.

Códigos de error

Cada elemento de `errores` trae `campo`, `codigo`, `detalle` y `mensaje`. Si aplica, también `tipo_esperado` y `unidad_esperada`. El valor recibido no se copia a un campo de la respuesta que no acepte ese tipo.

Código	Cuándo
`TIPO_INVALIDO`	El dato no es del tipo del contrato
`UNIDAD_AUSENTE` / `UNIDAD_INCOMPATIBLE`	Falta la unidad o no es la canónica. No hay conversión
`VARIABLE_DUPLICADA` / `CLAVE_JSON_DUPLICADA`	La misma variable o la misma clave JSON aparece más de una vez
`PARAMETRO_PROTEGIDO`	Se intenta cambiar la ficha en modo operacional
`ENTRADA` inválida por `LIMITE_TAMANO`	El JSON, la profundidad o el número de elementos supera el límite y no se recorre
`DATASET_DESCONOCIDO` / `VERSION_INCOMPATIBLE`	Dataset o contrato que este motor no evalúa
`FISICAMENTE_INADMISIBLE` / `VALOR_NO_FINITO`	Dominio de la fórmula, no un límite operacional aprobado
`ERROR_INTERNO`	Fallo imprevisto. `incidente_id` identifica el registro interno. No hay recomendación

Una regla `pendiente_validacion` es una observación preliminar. `no_evaluable` significa que falta evidencia, no que el pozo esté bien ni mal. No se completa una medición ausente con un promedio.

Las plantillas ejecutables están en `ejemplos/plantillas/`. Cada archivo trae la solicitud y el resultado que produjo este motor. Los números de QF-084 son los registrados en la hoja, no una recomendación.

Desde Python, en la raíz del expediente:

```
from evaluacion.motor import evaluar_solicitud
evaluar_solicitud(solicitud)
```

Errores y datos insuficientes

Un error de validación y un caso no evaluable no se presentan igual. El primero rechaza la solicitud. El segundo acepta la solicitud y dice qué falta. Los dos dejan la recomendación operacional en falso.

La evidencia de la comparación sin escalar viaja dentro de la regla, marcada como no vigente, para que se pueda auditar el artefacto sin volver a usarlo como resultado.

Qué puede hacer la aplicación

Puede listar los pozos de `pozos.json`, mostrar los parámetros registrados, enviar una corrección con unidad y pintar el JSON de salida campo por campo. Puede guardar la versión del motor y de la regla que aparecen en la respuesta.

Qué no puede hacer

- Inventar un límite de frecuencia o un límite de fabricante.
- Tratar una regla pendiente como si estuviera aprobada.
- Ordenar un cambio de frecuencia, de etapas o de tubing a partir de RC-BES-01, 02 o 03.
- Declarar que el pozo está sano porque el caudal quedó entre los números de la hoja.
- Usar `=TODAY()` como fecha de la medición.
- Usar el nombre de la hoja como identificador global definitivo.
- Llamar a un modelo de lenguaje para que calcule o apruebe.
- Leer `reports/figuras/caudal_y_limites.png` ni `head_tabulado.png`: son figuras viejas. El linaje está en `docs/linaje_artefactos.md`.

Modelo de lenguaje, más adelante

`capa_linguistica.md` dice qué JSON puede recibir y qué no puede redactar. Hasta que esa capa exista, la aplicación usa los campos `mensaje` de la respuesta. Esos textos ya distinguen observación, advertencia y condición no evaluable.

Qué probar antes de usarlo

```
PYTHONPATH=src /home/antonio/projects/HUC/.venv/bin/python -m evaluacion.autotest
```

Ese comando corre pytest sobre `tests/` y escribe `logs/autotest_reporte.json` y `logs/autotest_reporte.md`. Tiene que quedar en estado `aprobado`. En particular, con los datos registrados de QF-084 el resultado es RC-BES-03 y `recomendacion_operacional_autorizada` es falso.

Cómo se regenera este paquete

```
PYTHONPATH=src /home/antonio/projects/HUC/.venv/bin/python -m
integracion.construir_paquete
```

Ese comando reescribe los JSON, esta guía, la ficha maestra y el catálogo de reglas. No mueve el expediente a `procesadas`.